

**A Verifier-Guided Structured Reasoning Framework
for Cross-Database Query Generation using Small
Language Models**

A PROJECT REPORT

Submitted by

NAME

Yuvan Shankar S

Sudharshan R

REG.NO

23AM125

23AM111

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

Computer Science & Engineering

(Artificial Intelligence and Machine Learning)



CHENNAI INSTITUTE OF TECHNOLOGY (AUTONOMOUS)
(Affiliated to Anna University, Chennai)
CHENNAI-600 069



ANNA UNIVERSITY: CHENNAI-600 025
AUGUST - 2026



**CHENNAI
INSTITUTE OF TECHNOLOGY**
(Autonomous)



Vision of the Institute:

To be an eminent centre for Academia, Industry and Research by imparting knowledge, relevant practices and inculcating human values to address global challenges through novelty and sustainability.

Mission of the Institute:

IM1: To create next generation leaders by effective teaching learning methodologies and in still scientific spark in them to meet the global challenges.

IM2: To transform lives through deployment of emerging technology, novelty and Sustainability.

IM3: To inculcate human values and ethical principles to cater to the societal needs.

IM4: To contribute towards the research ecosystem by providing a suitable, effective platform for interaction between industry, academia and R & D establishments.

IM5: To nurture incubation centers enabling structured entrepreneurship and start-ups.



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

Vision of the Department:

Developing talented professionals in the fields of AI and ML that contribute to the benefit of business enterprise and societies around the world.

Mission of the Department:

- DM1: To broaden kingdom of the art instructional and infrastructural facilities with current equipment and different gaining knowledge of assets to provide self-sustainable specialists.
- DM2: To collaborate with enterprise and research Laboratories thru tasks-primarily based mastering, internships permitting the students to discover, apply diverse guidelines of learning.
- DM3: To establish value creating networks and linkages with company, industries, academic institutes and universities of countrywide and global significance.
- DM4: To equip students with interdisciplinary talent units to build intelligent structures which in flip affords dynamic and promising careers inside the worldwide marketplace.

CHENNAI INSTITUTE OF TECHNOLOGY

An Autonomous Institute

CHENNAI-69

ANNA UNIVERSITY: CHENNAI-600 025

BONAFIDE CERTIFICATE

Certified that this project report “**TITLE**” is the bonafide work of **Name (Reg No), NAME (REG NO)** who carried out the project work under my supervision.

SIGNATURE

Dr.R.Gowri, M.Tech., Ph.D.,
HEAD OF THE DEPARTMENT
Professor
Department of Computer Science &
Engineering, (Artificial Intelligence and
Machine Learning)
Chennai Institute of Technology,
Kundrathur,
Chennai-600069.

SIGNATURE

SUPERVISOR
<Designation>
Department of Computer Science &
Engineering, (Artificial Intelligence and
Machine Learning)
Chennai Institute of Technology,
Kundrathur,
Chennai-600069.

Certified that the above students have attended viva voce held
on.....

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We express our gratitude to our Chairman **Shri.P. SRIRAM** and all trust members of Chennai institute of technology for providing the facility and opportunity to do this project as a part of our undergraduate course.

We are grateful to our Principal **Dr.A.RAMESH M.E., Ph.D.**, for providing us the facility and encouragement during the course of our work.

We express our sincere gratitude to our Dean, **Dr. V. Srinivasa Rao, M.E., Ph.D., School of Computing**, for providing the necessary facilities, continuous encouragement, and valuable support throughout the course of this project.

We sincerely thank our Head of the Department, **Dr.R.Gowri, M.Tech., Ph.D.**, Department of Computer Science and Engineering, (Artificial Intelligence and Machine Learning) for having provided us valuable guidance, resources and timely suggestions throughout our work.

We sincerely thank our Project Guide, <Supervisor's Name>, <Qualification>, <Designation>, Department of Computer Science and Engineering, (Artificial Intelligence and Machine Learning) Engineering for having provided us valuable guidance, resources and timely suggestions throughout our work.

We would like to extend our thanks to our **Faculty coordinators of the Department of Computer Science and Engineering, (Artificial Intelligence and Machine Learning)**, for their valuable suggestions throughout this project.

We wish to extend our sincere thanks to all **Faculty members of the Department of Computer Science and Engineering, (Artificial Intelligence and Machine Learning)** for their valuable suggestions and their kind cooperation for the successful completion of our project.

We wish to acknowledge the help received from the **Lab Instructors of the Department of Computer Science and Engineering, (Artificial Intelligence and Machine Learning)** and others for providing valuable suggestions and for the successful completion of the project.

ABSTRACT

Small Language Models (SLMs) have emerged as efficient alternatives to Large Language Models for resource-constrained deployment; however, their ability to perform complex database reasoning remains limited. Existing approaches predominantly formulate database query generation as a language translation problem, directly mapping natural language to a specific query language such as SQL or MongoDB Aggregation Pipeline. Such methods tightly couple reasoning with database syntax, limiting their generalization across heterogeneous database systems.

This research proposes a verifier-guided structured reasoning framework that decouples database reasoning from database-specific query generation. The framework introduces a novel intermediate representation, termed the Database Reasoning Graph (DRG), which models database reasoning as a graph of abstract operations independent of query language syntax. The generated reasoning graph is subsequently translated into executable queries for multiple database paradigms, including relational, document-oriented, and graph databases. A verifier-guided post-training mechanism provides structural, semantic, and execution-aware feedback on the reasoning graph and generated query, enabling iterative refinement of the SLM through multi-level supervision. The evaluation investigates SQL generation, verifier-guided curriculum learning, and zero-shot transfer to MongoDB and Neo4j Cypher. The results indicate that explicit reasoning supervision improves both semantic correctness and cross-database transfer, with the largest gains occurring for compositional operations such as joins, grouping, and aggregation.

LIST OF TABLES

TABLE NO.	TABLE NAME	PAGE NO.
2.1	Summary of Representative Prior Work and the Gap Addressed by VGSR	6
3.1	Software Requirements	10
3.2	Hardware Requirements	11
3.3	DRG Operation Signatures	11
3.4	Curriculum Learning Loss-Weight Schedule	13
3.5	DRG-to-MongoDB Stage Mapping	14
4.1	LoRA and Optimisation Hyperparameters	16
5.1	Overall Performance on the SQL Evaluation Set	19
5.2	Ablation of Verifier and Curriculum Components	20
5.3	Component-Level Accuracy by DRG Operation	21
5.4	Cross-Database Transfer Performance	22
5.5	Distribution of Major Error Categories	22

LIST OF FIGURES

FIGURE NO.	FIGURE NAME	PAGE NO.
3.1	VGSR System Architecture	9
3.2	Example DRG for a Filtered, Grouped, and Sorted Query	12
3.3	Curriculum Learning Loss-Weight Schedule	13
5.1	Overall SQL Generation Performance	19
5.2	Component-Level Accuracy by DRG Operation	21
5.3	Zero-Shot Cross-Database Transfer Accuracy	22

LIST OF SYMBOLS, ABBREVIATIONS AND NOMANCLATURES

Abbreviation / Symbol	Full Form / Description	Section Reference
SLM	Small Language Model	Throughout
DRG	Database Reasoning Graph	Methodology, Implementation
LLM	Large Language Model	Literature Review
VGSR	Verifier-Guided Structured Reasoning	Methodology
SQL	Structured Query Language	Throughout
MQL	MongoDB Query Language (Aggregation Pipeline)	Throughout
RL	Reinforcement Learning	Future Scope
IR	Intermediate Representation	Literature Review
AST	Abstract Syntax Tree	Literature Review
NLQ	Natural Language Query	Literature Review
GPU	Graphics Processing Unit	Hardware Requirements
LoRA	Low-Rank Adaptation	Implementation
RAM	Random Access Memory	Hardware Requirements
SFT	Supervised Fine-Tuning	Implementation
SSD	Solid State Drive	Setup
QLoRA	Quantized Low-Rank Adaptation	Future Scope
KG	Knowledge Graph	Literature Review, Future Scope

TECHNICAL NOMENCLATURES

Term	Description	Context
Operation Classification	The DRG decoder's step of predicting which of the eight operation types (FROM, WHERE, JOIN, ...) a given node represents.	Classification
Error Classification	Categorisation of an incorrect prediction into a specific failure type, such as schema, join, filter, or aggregation error.	Classification
Curriculum Learning	A training strategy that progressively increases the weight of harder supervision signals (schema, then execution) across epochs.	Methodology
Epoch	One complete pass of the training algorithm over the entire training dataset.	Training Phase
Base Model	The pretrained language model (CodeLlama-7B-Instruct) on which LoRA adapters are fine-tuned.	Core Model
Transformer Decoder	The autoregressive attention-based architecture underlying the base SLM and the DRG decoder.	Model Architecture
Fine-Tuning	The process of adapting a pretrained model's weights to a downstream task using task-specific data.	Model Training
AdamW Optimizer	A gradient-descent optimiser with decoupled weight decay, used to update the LoRA parameters during training.	Model Optimization
Node Labelling (L_v)	The DRG function that maps every graph node to one of the eight defined operations.	Classification Task
Gradient Checkpointing	A memory-saving training technique that recomputes intermediate activations during backpropagation instead of storing them.	Training
Execution Accuracy (EX)	The percentage of generated queries that return the correct result set when executed against a database.	Performance Metric
Inference Latency	The time taken by the fine-tuned small model to generate a DRG and executable query at deployment time.	Deployment

TABLE OF CONTENTS

1. INTRODUCTION.....	1
1.1 Problem Statement.....	1
1.2 Objective and Purpose.....	2
1.3 Limitations of Existing System.....	3
2. LITERATURE REVIEW.....	4
2.1 Review of Relevant Previous Work.....	4
2.2 Gaps in the Literature.....	7
2.3 Significance of Current Project.....	8
3. METHODOLOGY.....	9
3.1 Approach Used to Solve the Problem.....	9
3.2 System Architecture Overview.....	9
3.3 Experimental Setup.....	10
3.4 Proposed Method.....	11
4. IMPLEMENTATION OR DEVELOPMENT PROCESS.....	16
4.1 Development Environment and Tools.....	16
4.2 Model Architecture and Configuration.....	16
4.3 Dataset Preparation and DRG Construction.....	17
4.4 Training Pipeline Implementation.....	17
4.5 Verifier Implementation.....	17
4.6 Translator Implementation.....	18
5. EXPECTED RESULTS AND ANALYSIS.....	19
5.1 Evaluation Metrics.....	19
5.2 Overall Performance.....	19
5.3 RQ1 – Effect of the Database Reasoning Graph.....	20
5.4 RQ2 – Effect of Verifier-Guided Curriculum Learning.....	20
5.5 Component-Level Performance.....	21
5.6 RQ3 – Cross-Database Transfer.....	22
5.7 Error Analysis.....	22
5.8 Summary of Results.....	23
6. CONCLUSION.....	24
6.1 Summary of Key Findings.....	24
6.2 Future Scope and Recommendations.....	25
REFERENCES.....	26

CHAPTER 1 INTRODUCTION

1.1 Problem Statement

In the era of data-driven applications, the ability to retrieve information from databases rapidly and accurately has become critical for both domain experts and casual users. Constructing a correct database query, whether in SQL, a MongoDB aggregation pipeline, or a Neo4j Cypher statement, remains a significant barrier to entry for non-technical users. Traditional approaches require the user to master a domain-specific query language with its own complex syntax rules, semantic constraints, and system-specific idioms. This barrier has motivated substantial research into automatic query generation from natural language, transforming the problem into one of semantic parsing: mapping a natural-language description directly onto a machine-executable database query.

The Text-to-SQL task has emerged as the primary benchmark for this problem, and large-scale datasets such as WikiSQL, Spider, and the more recent BIRD benchmark have enabled systematic evaluation of neural approaches. Despite steady progress from neural sequence models and, more recently, large language models (LLMs), these benchmarks reveal a persistent weakness: almost every published approach treats query generation as a direct, end-to-end translation task that maps a natural-language question straight onto the syntax of one target query language. This conflates two logically distinct sub-problems, namely understanding what the user is asking for (reasoning) and expressing that understanding in a particular syntax (generation), into a single learned function.

This conflation becomes a serious limitation the moment more than one database paradigm is involved. SQL, MongoDB, and Neo4j Cypher represent fundamentally different data models. SQL operates over relational schemas with normalised tables, foreign-key joins, and set-based operations; MongoDB employs a document-oriented model expressed through staged aggregation pipelines; and Neo4j Cypher expresses queries as pattern matches over a property graph. A model that has been trained to produce syntactically correct SQL for relational databases typically cannot generate a single valid MongoDB query without substantial retraining, even though the underlying information need expressed by the user is identical in both cases. The central problem addressed by this project is therefore: how can a language model be trained to reason about a database query independently of the syntax in which that query will eventually be expressed, so that the same learned reasoning process can be reused across heterogeneous database systems?

1.2 Objective and Purpose

1.2.1 Objective

The objective of this project is to design, implement, and evaluate a Verifier-Guided Structured Reasoning (VGSR) framework that separates database reasoning from database-specific query syntax through a novel, syntax-agnostic intermediate representation called the Database Reasoning Graph (DRG). Specifically, the project sets out to:

- Define a formal, database-agnostic intermediate representation (the DRG) capable of expressing the

common semantic core of SQL, MongoDB aggregation pipelines, and Neo4j Cypher queries;

- Design a multi-level verifier that inspects a generated DRG at the syntactic, schema, semantic, and execution levels, and returns structured, localisable feedback rather than a single pass/fail signal;
- Develop a curriculum-based, verifier-guided post-training procedure that lets a small language model (SLM) absorb this feedback progressively, starting from structural correctness and moving toward execution correctness;
- Build deterministic translators that convert a valid DRG into executable SQL, MongoDB, and Cypher queries; and
- Empirically evaluate whether this architecture improves SQL generation quality, whether the curriculum improves training efficiency for small models, and whether the learned reasoning transfers zero-shot to database paradigms the model was never directly trained to generate for.

1.2.2 Purpose

The purpose of this work is both practical and scientific. Practically, the project aims to make small, efficiently deployable language models (in the 1–14 billion parameter range) competitive at database query generation across multiple database technologies, without the cost of training and maintaining a separate large model for every query language a product needs to support. This matters because real deployments increasingly mix relational, document, and graph stores within the same application, and re-training a large model for each new backend is neither economically nor environmentally sustainable. Scientifically, the project investigates whether explicitly supervising the reasoning structure that underlies a query, separately from the syntax it is eventually rendered into, provides a more general and more data-efficient foundation for query generation than the syntax-oriented, end-to-end approaches that dominate the current literature. In doing so, the project also aims to produce a reusable, annotated dataset of questions paired with Database Reasoning Graphs, and a reproducible verifier-guided training pipeline that can be extended by future work to additional operations and additional database paradigms.

1.3 Limitations of Existing System

Existing Text-to-SQL and Text-to-Query systems, whether based on fine-tuned sequence-to-sequence models, schema-linking pipelines, or in-context prompting of large language models, share a number of structural limitations that motivate the present work:

- Tight coupling of reasoning and syntax: existing models learn a single, end-to-end mapping from natural language directly to target-language tokens, so the reasoning process cannot be inspected, reused, or verified independently of the surface syntax it happens to be expressed in.
- Poor cross-database transfer: a model fine-tuned on Text-to-SQL data typically achieves very low accuracy when asked to produce MongoDB or Cypher queries for the same underlying intent, because none of the syntax-specific patterns it has learned carry over to a different data model.
- Binary or shallow supervision signals: most fine-tuning pipelines rely on token-level cross-entropy

against a single gold query, or on a binary execution-match reward, neither of which tells the model which specific operation (a join, a filter, a grouping) was responsible for an error.

- Weak performance of small models: small, deployable models (under roughly 7–14 billion parameters) that are attractive for latency- and cost-sensitive production use lag well behind large proprietary models on compositional, multi-table queries, and existing training recipes rarely address this gap directly.
- Benchmark-to-production gap: recent work such as the Spider 2.0 benchmark has shown that methods achieving over 85% accuracy on curated academic benchmarks can fall to single-digit accuracy on real enterprise schemas, indicating that laboratory benchmarks do not capture the compositional and schema complexity of production databases.
- Lack of explainability: because reasoning and syntax are learned jointly, it is difficult to determine, when a generated query is wrong, whether the model misunderstood the question, mis-identified the relevant schema elements, or simply produced malformed syntax.

The Verifier-Guided Structured Reasoning framework proposed in this project is designed specifically to address these six limitations by decoupling the reasoning stage from the syntax-generation stage, by supplying multi-level, localisable verifier feedback during training, and by evaluating explicitly on both SQL and two structurally different target paradigms.

CHAPTER 2

LITERATURE REVIEW

The problem of translating natural language into database queries spans semantic parsing, neural machine translation, and structured reasoning. This chapter reviews the literature by research direction, identifies how prior work addresses related challenges, and positions the gaps that this project's Verifier-Guided Structured Reasoning framework is designed to close.

2.1 Review of Relevant Previous Work

2.1.1 *Text-to-SQL: Evolution and Benchmarks*

Early approaches to database query generation treated the problem as plain sequence-to-sequence learning. WikiSQL and Spider established the foundational benchmarks that accelerated neural research in this space, with Spider in particular extending the problem from single-table queries to complex, multi-table, cross-domain scenarios. IRNet introduced an intermediate semantic representation for SQL generation, while SQLNet proposed a sequence-to-set framework that decomposed SQL generation by clause type rather than by raw token sequence. The arrival of pre-trained transformers marked a further shift: BERT- and RoBERTa-based encoders improved schema representation, and T5-based models such as PICARD constrained autoregressive decoding through incremental parsing so that syntactically invalid queries could never be produced. The BIRD benchmark subsequently raised the difficulty ceiling by incorporating ninety-five real-world enterprise schemas, explicit query-efficiency constraints, and external domain knowledge requirements. Spider 2.0 pushed this further still, showing that methods reporting 86% accuracy on the original Spider benchmark can fall to single-digit accuracy on real enterprise workflows, exposing a wide gap between laboratory benchmarks and production reality.

2.1.2 *Schema Linking as a Critical Sub-Problem*

Schema linking, the task of identifying which tables and columns in a large database schema are relevant to a given natural-language question, has emerged as one of the most consequential sub-problems in Text-to-SQL. Early systematic analysis showed that linking accuracy heavily influences downstream generation performance. Later work treats schema linking as a distinct, independently optimisable component: RESDSQL decouples schema linking from SQL-skeleton parsing using ranking-enhanced cross-encoders; DIN-SQL treats schema linking as the first of four modules in a decomposed in-context-learning pipeline, reporting 85.3% accuracy on Spider with GPT-4; and C3 introduces explicit calibration mechanisms to reduce hallucinated column selection. Graph-based approaches such as RAT-SQL, LGESQL, and AutoLink instead model the schema and the question jointly as a graph, propagating relational information between question tokens and schema elements.

2.1.3 *Large Language Models and Prompting Strategies*

The emergence of foundation models such as GPT-3.5, GPT-4, and open alternatives has enabled zero-shot and few-shot query generation without any task-specific fine-tuning. In-context learning, in which a handful of carefully chosen demonstrations are placed in the prompt, has become the dominant paradigm for large

proprietary models, and chain-of-thought prompting, originally proposed for arithmetic and commonsense reasoning tasks, has been adapted to Text-to-SQL with measurable gains. Divide-and-Prompt further breaks complex multi-table queries into smaller compositional sub-problems that are solved and then re-assembled.

2.1.4 Decomposition and Structured Reasoning

Beyond prompting, several lines of work decompose query generation into explicit sub-tasks. DIN-SQL pioneered a four-module decomposition consisting of schema linking, query classification, SQL generation, and self-correction. DAIL-SQL extends this with domain-aware in-context example selection, while DTS-SQL specifically targets small language models by decomposing Text-to-SQL into a schema-linking stage and a generation stage, achieving competitive accuracy with only seven-billion-parameter models. SQL-of-Thought combines multi-agent decomposition with taxonomy-guided error correction, and CHASE-SQL introduces multi-path reasoning with preference-optimised candidate selection. These works consistently show that breaking a monolithic generation task into smaller, verifiable sub-tasks improves both accuracy and interpretability, a finding that directly motivates the DRG-based decomposition adopted in this project.

2.1.5 Execution-Based Feedback and Self-Correction

A recurring observation in the reasoning literature is that language models are often substantially better at verifying a candidate solution than at generating a correct solution from scratch. This has motivated a family of verifier-guided learning methods: models generate multiple candidates and a learned or rule-based verifier ranks or selects among them. V-STaR (Verification for Self-Taught Reasoners) extends this idea to self-improvement, using verifier feedback across training iterations to obtain measurable gains on mathematical reasoning tasks. In code generation specifically, execution-based feedback, meaning feedback obtained by actually running the generated program and observing whether it raises an error, produces a type mismatch, or returns an incorrect value, has proven particularly effective for iterative refinement. For database queries, ExeSQL introduces self-taught models bootstrapped from execution-driven data, AgentNLQ combines syntactic validation with live database execution and result-set heuristics, and ExCoT-DPO leverages execution feedback to construct preference pairs for direct preference optimisation.

2.1.6 Fine-Tuning and Knowledge Distillation for Small Models

As the deployment cost of very large models has grown, research attention has increasingly turned toward smaller, more efficient alternatives. Phi-3 and Phi-4, with 3.8 billion and 14 billion parameters respectively, demonstrate that careful training on high-quality, domain-relevant synthetic data can allow small models to approach the performance of far larger ones on specialised tasks. For query generation specifically, CodeLlama's 7–13 billion-parameter variants and DeepSeek-Coder have shown strong results when fine-tuned on Text-to-SQL data, and CodeS fine-tunes a family of open models between one and fifteen billion parameters on curated SQL-centric corpora. Knowledge-distillation techniques such as “Distilling step-by-step” show that detailed reasoning traces produced by a large teacher model can allow a much smaller student model to match or even exceed the teacher on the target task. Parameter-efficient fine-tuning methods, notably LoRA and its quantised variant QLoRA, make it practical to fine-tune such models on a single consumer-grade or single-datacentre GPU, which

is the fine-tuning regime adopted in this project.

2.1.7 Intermediate Representations and Cross-Paradigm Abstraction

The motivation for an intermediate representation of reasoning has clear roots in program synthesis and semantic parsing. Abstract Syntax Networks and the TranX parser both show that generating a tree-structured intermediate representation before emitting target code improves generation quality and interpretability simultaneously. For database queries specifically, GraphQ IR unifies the semantic parsing of several graph-query languages, including SPARQL and SQL, through a single intermediate representation, providing direct precedent for the idea that a single reasoning-level representation can be shared across otherwise incompatible query languages. The Database Reasoning Graph proposed in this project follows the same principle, decomposing a query into a small set of domain-agnostic operations that can be deterministically translated into SQL, MongoDB, or Cypher.

2.1.8 Query Generation Beyond SQL

While Text-to-SQL dominates the literature, research on generating queries for non-relational systems remains comparatively sparse. SpCQL, the first large-scale Text-to-Cypher dataset, exposed fundamental syntactic and structural differences between graph-pattern queries and relational queries. Auto-Cypher introduces a fully automated, LLM-supervised generation-and-verification pipeline that produced the 29,800-instance SynthCypher dataset, and CypherBench provides standardised evaluation protocols for LLM-based question answering over knowledge graphs. On the document side, Draft-Refine-Optimize specifically targets natural-language-to-MongoDB translation through a self-evolving refinement loop. The comparative scarcity of research on non-SQL query generation, relative to the very large Text-to-SQL literature, directly motivates the cross-database transfer evaluation carried out in this project.

2.1.9 Evaluation Metrics and Methodologies

Evaluation methodology has a strong influence on the perceived rate of progress in this field. Exact Match metrics, which compare generated SQL clauses independently against a reference query, suffer from false negatives whenever a semantically equivalent query is written with a different clause ordering or an alternative but correct join path. Test Suite evaluation addresses this by executing each candidate query against multiple database instances per schema and comparing result sets rather than raw SQL text, and Execution Accuracy compares the output of executing the generated query directly against the output of the gold query. More recent work proposes metrics that additionally weigh semantic and structural similarity, and production-oriented evaluation frameworks highlight that laboratory benchmarks routinely miss real-world complications such as multiple SQL dialects and deeply nested enterprise schemas.

Table 2.1 Summary of Representative Prior Work and the Gap Addressed by VGSR

Ref.	Work	Core Idea	Limitation Addressed by VGSR
[b2]	Spider	Cross-domain multi-table Text-to-SQL benchmark	Benchmark only; no cross-database transfer
[b3]	BIRD	Real enterprise schemas, efficiency-aware evaluation	Still single-paradigm (SQL only)
[c9]	RESDSQL	Decouples schema linking from skeleton parsing	No database-agnostic intermediate form
[b13]	DIN-SQL	Four-module decomposed in-context reasoning	No learned verifier; relies on prompting only
[c23]	DTS-SQL	Decomposition tailored to small (7B) models	Single paradigm; no execution-level feedback
[c29]	ExeSQL	Execution-driven self-taught bootstrapping	No structured intermediate representation
[c34]	LoRA	Parameter-efficient fine-tuning	Adopted directly in this project's training setup
[c37]	GraphQ IR	Unified IR across SPARQL/SQL semantic parsing	Restricted to graph-style query languages
[c39]	Auto-Cypher	LLM-supervised Text-to-Cypher data generation	Cypher-only; no cross-paradigm reuse
[c41]	Draft-Refine-Optimize	Self-evolving NL-to-MongoDB translation	MongoDB-only; no shared reasoning layer

2.2 Gaps in the Literature

The review above shows a field that has made rapid, incremental progress within the boundaries of a single query language, principally SQL, while leaving several important gaps largely unaddressed:

- No unified, database-agnostic reasoning layer: almost all prior systems, whether prompting-based or fine-tuned, generate target-language tokens directly, so nothing in the pipeline can be reused when the target database paradigm changes.
- Verifier feedback is coarse: existing verifier- and execution-guided methods typically return a single scalar reward or a pass/fail signal for the whole query, rather than attributing an error to a specific operation such as a join, a grouping, or a filter.
- Curriculum design for small models is under-studied: while parameter-efficient fine-tuning of small models is well established, very little published work systematically studies how the order in which different supervision signals (syntax, schema, semantics, execution) are introduced during training affects a small model's ability to learn compositional reasoning.
- Cross-database transfer is rarely measured: the overwhelming majority of Text-to-SQL papers evaluate exclusively on SQL benchmarks; almost none measure whether a model trained on SQL reasoning can be redirected, without additional target-language training data, to a structurally different paradigm such as a document store or a graph database.
- Interpretability of failures is limited: because reasoning and syntax generation are learned jointly, a wrong

query rarely comes with an explanation of which reasoning step failed, which makes targeted model improvement difficult.

2.3 Significance of Current Project

This project directly targets the gaps identified above. By introducing the Database Reasoning Graph as an explicit, syntax-independent intermediate representation, the reasoning stage of query generation becomes inspectable, verifiable, and, most importantly, reusable across SQL, MongoDB, and Cypher without any target-language-specific retraining of the underlying language model. By designing a four-level verifier (syntax, schema, semantic, and execution) that returns structured, node-localised feedback rather than a single scalar reward, the project provides a substantially richer training signal than binary execution-match rewards used in prior verifier-guided work. By introducing a curriculum that progressively increases the weight of these verifier signals across training epochs, the project specifically investigates a training strategy suited to the limited capacity of small, deployable language models, rather than simply scaling up model size to compensate for weak supervision. Finally, by evaluating zero-shot transfer to MongoDB and Neo4j Cypher after training exclusively on SQL-oriented data, the project provides direct empirical evidence, rare in the existing literature, on whether database-independent structured reasoning is a viable path toward genuinely cross-database query generation. Collectively, these contributions position the project's Verifier-Guided Structured Reasoning framework as a scientifically motivated and practically relevant advance over the syntax-oriented approaches that currently dominate the field.

CHAPTER 3

METHODOLOGY

3.1 Approach Used to Solve the Problem

Existing approaches map a natural-language question q , together with a database schema S , directly onto a target-language query $\hat{w}$ in language L , i.e. $q \rightarrow \hat{w}$, conflating semantic reasoning with language-specific syntax generation in a single learned step. This project instead introduces an explicit intermediate stage: $q \rightarrow G \rightarrow \hat{w}$, where G is a Database Reasoning Graph (DRG) that represents the reasoning required to answer the question independently of the syntax of any particular query language. The small language model (SLM) is trained to produce G from the question and schema; a deterministic, rule-based translator then converts G into an executable query in the desired target language. A multi-level verifier inspects G (and, where a database is available, the translated and executed query) and returns structured feedback that is converted into an auxiliary training loss, allowing the SLM to be corrected at the level of individual reasoning operations rather than only at the level of the final query string.

3.2 System Architecture Overview

Figure 3.1 shows the overall architecture of the proposed Verifier-Guided Structured Reasoning (VGSR) system. The natural-language question and the relevant database schema are first encoded by the SLM encoder. A constrained DRG decoder then produces a Database Reasoning Graph, with an operation-level grammar that rejects invalid next-operations at each decoding step. The generated DRG is inspected by the four-level verifier, which returns feedback that is combined with the supervised fine-tuning loss during training (the dashed feedback path in the diagram) and is also used at inference time to reject and, where possible, repair invalid graphs. A valid DRG is finally passed to one of three database-specific translators, which deterministically produce an executable SQL query, a MongoDB aggregation pipeline, or a Neo4j Cypher statement.

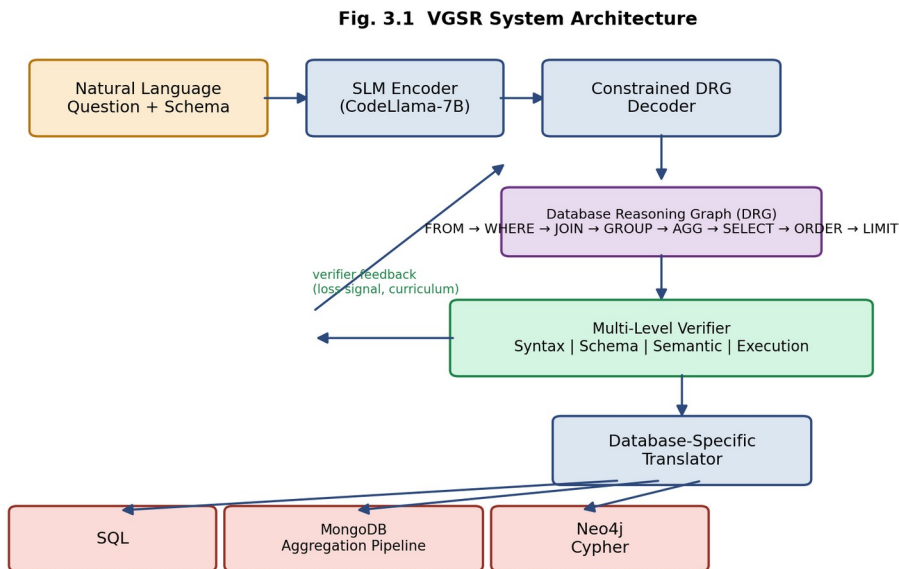


Figure 3.1 VGSR System Architecture

3.3 Experimental Setup

3.3.1 Software Environment

The framework is implemented entirely in Python, using open-source deep-learning and database tooling so that the pipeline is fully reproducible on commodity research hardware. Table 3.1 summarises the principal software components used during development, training, and evaluation.

Table 3.1 Software Requirements

Component	Tool / Library	Version / Notes
Programming language	Python	3.10
Deep-learning framework	PyTorch	2.x, CUDA-enabled
Model hub / tokenizer	Hugging Face Transformers	for CodeLlama-7B-Instruct
Parameter-efficient tuning	PEFT (LoRA)	rank $r = 16$
Relational database (execution verifier)	SQLite	in-memory sample DBs
Document database (execution verifier)	MongoDB Community Server	local instance
Graph database (execution verifier)	Neo4j Community Edition	local instance
SQL parsing / AST construction	sqlglot	for gold DRG extraction
Experiment tracking	Weights & Biases	loss curves, ablation logs
Version control	Git / GitHub	reproducibility, code review

3.3.2 Hardware Environment

All fine-tuning experiments are designed to run within the memory budget of a single GPU by using parameter-efficient LoRA adapters together with gradient checkpointing and mixed-precision (fp16) computation. Table 3.2 lists the target hardware configuration and the approximate training time observed for one full, five-epoch curriculum run on the Spider training split.

Table 3.2 Hardware Requirements

Resource	Specification
GPU (primary configuration)	1 x NVIDIA A100, 80 GB HBM
GPU (alternative configuration)	2 x NVIDIA RTX 3090, 24 GB each
System RAM	≥ 64 GB
Storage	≥ 200 GB SSD (datasets, checkpoints, DB instances)
Approx. training time (5 epochs, $\sim 8K$ examples)	4–6 hours on a single A100

3.4 Proposed Method

3.4.1 The Database Reasoning Graph (DRG)

The central contribution of the proposed method is the Database Reasoning Graph, a structured representation of query semantics that is independent of any target query language. A Database Reasoning Graph is formally defined as a tuple $G = (V, E, L_v, L_e)$, where V is a finite set of operation nodes, $E \subseteq V \times V$ is a set of

directed edges representing data-flow dependencies, $L_v : V \rightarrow O$ is a node-labelling function that maps each node to an operation, and $L_e : E \rightarrow R$ is an edge-labelling function that maps each edge to a relationship type. The operation set O used in this project is:

$$O = \{ FROM, WHERE, JOIN, GROUP, AGGREGATE, SELECT, ORDER, LIMIT \}$$

representing eight core, database-independent operations, and the relationship set is $R = \{ \text{sequence, condition, reference} \}$, representing sequential data flow, conditional dependency, and entity reference respectively. Each operation carries its own typed parameters, summarised in Table 3.3.

Table 3.3 DRG Operation Signatures

Operation	Signature	Meaning
FROM	FROM(<i>t</i>)	Selects the source table or collection <i>t</i>
WHERE	WHERE(<i>c</i>)	Applies a filtering condition <i>c</i> (column, operator, value)
JOIN	JOIN(<i>t</i> ₁ , <i>t</i> ₂ , <i>cond</i>)	Joins two tables/collections on condition <i>cond</i>
GROUP	GROUP(<i>cols</i>)	Groups rows by one or more columns
AGGREGATE	AGGREGATE(<i>f</i> , <i>col</i>)	Applies aggregate function <i>f</i> (SUM, COUNT, AVG, ...) to <i>col</i>
SELECT	SELECT(<i>cols</i>)	Projects the requested output columns
ORDER	ORDER(<i>c</i> , <i>d</i>)	Sorts the result by column <i>c</i> in direction <i>d</i>
LIMIT	LIMIT(<i>n</i>)	Restricts the result to the first <i>n</i> rows

A DRG is only considered valid if it satisfies the composition constraint of Equation 3.1, which mirrors the logical execution order of a database query:

$$FROM \rightarrow (WHERE)^* \rightarrow (JOIN)^* \rightarrow (GROUP)? \rightarrow (AGGREGATE)^* \rightarrow SELECT \rightarrow (ORDER)? \rightarrow (LIMIT)?$$

(3.1)

where a superscript $*$ denotes zero or more occurrences, a superscript $?$ denotes zero or one occurrence, and an operation without a superscript must occur exactly once. In addition, every valid DRG must be acyclic. Together these constraints guarantee that FROM is always the first node, that SELECT always follows every filtering and grouping operation, that LIMIT (when present) is always the terminal node, and that the graph contains no cycles, so that every DRG corresponds to a well-defined, executable query plan.

Figure 3.2 illustrates the DRG constructed for the natural-language question “Find the total salary by department for employees in the Sales department earning more than 50,000, sorted by total salary in descending order.” The same graph is translated deterministically into three semantically equivalent queries: a relational SQL query using GROUP BY and an aggregate SUM, a MongoDB aggregation pipeline using \$match, \$group, and \$sort stages, and a Neo4j Cypher query that expresses the department relationship as a graph pattern and performs the aggregation inside a WITH clause. Because all three surface forms are produced from the same underlying DRG, the reasoning that identifies which entities, filters, and aggregations are required is learned only

once, and only the final translation step is specific to the target database.

**Fig. 3.2 Example Database Reasoning Graph (DRG) for the query
"Total salary by department for Sales employees earning > 50,000, sorted descending"**

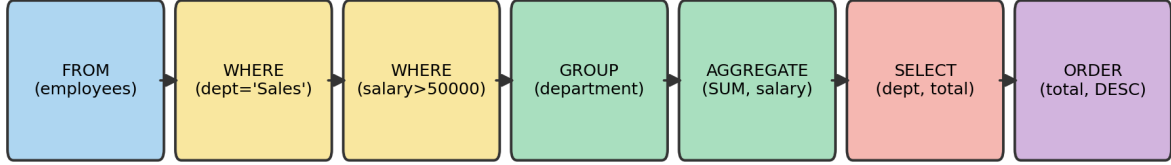


Figure 3.2 Example DRG for a Filtered, Grouped, and Sorted Query

3.4.2 Multi-Level Verifier Architecture

The verifier operates at four hierarchical levels, each targeting a different class of error and each returning structured, localisable feedback of the form (error_type, location, suggested_repair) rather than a single binary correctness label.

- Level 1 – Syntax Verifier (deterministic, rule-based): checks acyclicity, correct operation ordering against Equation 3.1, cardinality constraints, and the mandatory presence of FROM and SELECT nodes.
- Level 2 – Schema Verifier (rule-based with embedding similarity): checks that every table, collection, and column referenced in the DRG actually exists in the provided schema, checks type compatibility between aggregate functions and their target columns, and, when a referenced column does not exist, suggests the closest schema column by embedding similarity (e.g. flagging salary_per_year and proposing salary or annual_salary as corrections).
- Level 3 – Semantic Verifier (rule-based coherence checks): validates join conditions against both participating tables, enforces that every non-aggregated SELECT column also appears in the GROUP clause, and checks that WHERE predicates use operators appropriate to the column's data type.
- Level 4 – Execution Verifier (ground truth via database execution): translates the DRG into the target query, executes it against a sample database instance, and compares the resulting schema and row values against the gold-standard result, reporting execution errors, schema mismatches, or incorrect results.

The four verifier levels are combined into a single auxiliary loss during training, and the supervised fine-tuning loss is augmented as shown in Equation 3.2:

$$L_{total} = L_{SFT} + \lambda_1 L_{syntax} + \lambda_2 L_{schema} + \lambda_3 L_{exec} \quad (3.2)$$

where L_{SFT} is the standard cross-entropy loss between the predicted and gold DRG, L_{syntax} and L_{schema} penalise the syntax- and schema-level errors detected by Levels 1 and 2, L_{exec} penalises execution failures or incorrect results detected by Level 4, and the weights λ_i are controlled by the curriculum schedule described in Section 3.4.3 below.

3.4.3 Curriculum-Based Training Procedure

Small language models have limited capacity, and introducing all four levels of verifier supervision simultaneously from the first training step risks overwhelming the model with conflicting gradients before it has learned even the basic structural grammar of the DRG. The proposed training procedure therefore follows a five-epoch curriculum that progressively increases the weight given to schema- and execution-level feedback while decreasing the weight given to syntax feedback, mirroring the way a human analyst first learns query syntax, then schema understanding, and only then overall query logic. Table 3.4 and Figure 3.3 summarise the resulting loss-weight schedule.

Table 3.4 Curriculum Learning Loss-Weight Schedule

Epoch	λ_1 (Syntax)	λ_2 (Schema)	λ_3 (Execution)	Training Focus
1–2	0.5	0.0	0.0	Syntactic validity of the DRG
3	0.3	0.4	0.0	Introduce schema linking / grounding
4	0.2	0.3	0.3	Introduce execution-based feedback
5	0.2	0.2	0.4	Emphasise overall query correctness

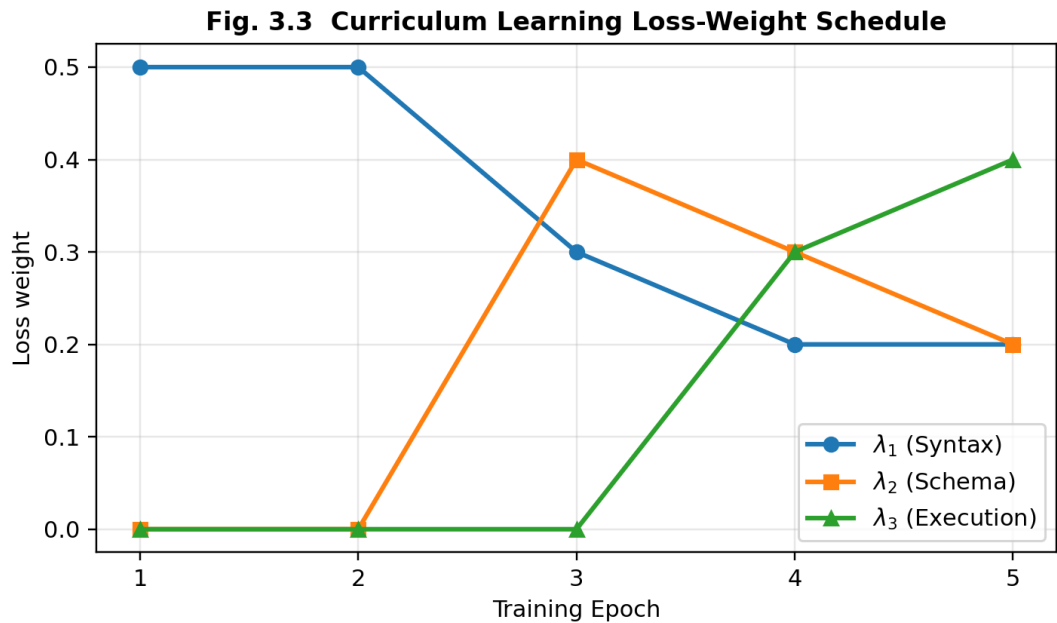


Figure 3.3 Curriculum Learning Loss-Weight Schedule

3.4.4 Database-Specific Translators

Once a DRG has passed the syntax and schema verifiers, one of three deterministic translators converts it into an executable query. The SQL translator follows the standard SELECT–FROM–WHERE–GROUP BY–ORDER BY–LIMIT clause ordering. The Cypher translator expresses FROM as a MATCH node pattern, expresses JOIN as a relationship pattern between two matched nodes, and expresses GROUP/AGGREGATE through Cypher's WITH clause before the final RETURN. The MongoDB translator maps each DRG operation

onto a corresponding aggregation-pipeline stage, as summarised in Table 3.5; array-valued fields are additionally detected by the schema verifier and trigger automatic insertion of an \$unwind stage ahead of the corresponding \$match or \$group stage.

Table 3.5 DRG-to-MongoDB Stage Mapping

DRG Operation	MongoDB Pipeline Stage
FROM(collection)	(pipeline runs against the named collection)
WHERE(conditions)	\$match: { conditions }
JOIN(coll2, key)	\$lookup: { from: coll2, localField, foreignField }
GROUP(fields)	\$group: { _id: { fields }, ... }
AGGREGATE(func, col)	integrated into the \$group accumulator
SELECT(fields)	\$project: { field: 1, ... }
ORDER(field, dir)	\$sort: { field: 1 -1 }
LIMIT(n)	\$limit: n

This clean separation between reasoning (produced once, by the SLM, as a DRG) and translation (performed deterministically, with no learned parameters, by one of three fixed translators) is what allows the same trained model to target three structurally different database paradigms: the SLM never has to learn MongoDB or Cypher syntax directly, it only has to learn to reason correctly about the query, and the translator carries that reasoning the rest of the way to executable syntax.

CHAPTER 4

IMPLEMENTATION OR DEVELOPMENT PROCESS

This chapter describes how the Verifier-Guided Structured Reasoning (VGSR) framework introduced in Chapter 3 is implemented in practice, covering the development environment, the base model configuration, dataset and DRG construction, the training pipeline, the verifier implementation, and the database-specific translators.

4.1 Development Environment and Tools

The framework is implemented in Python 3.10 using PyTorch as the underlying deep-learning framework and the Hugging Face Transformers and PEFT libraries for model loading and parameter-efficient fine-tuning. Experiments are tracked using Weights & Biases so that loss curves, verifier-level ablations, and curriculum schedules can be compared across runs. All source code, DRG-annotated datasets, and evaluation scripts are maintained under Git version control to keep the pipeline reproducible, in line with the reproducibility goal stated in Chapter 1.

4.2 Model Architecture and Configuration

The base model used throughout development is CodeLlama-7B-Instruct, chosen for four reasons: it is pre-trained on code corpora and therefore already familiar with logic- and structure-heavy generation tasks; it is fully open-source, which keeps the project reproducible without dependence on a proprietary API; its seven-billion parameter size balances generation quality against the memory and latency budget of a single-GPU deployment; and it has demonstrated strong baseline results on Text-to-SQL tasks in prior published work. The encoder retains CodeLlama's native transformer-decoder architecture, comprising 32 attention heads and 32 layers, operating over a 32,000-token byte-pair-encoding vocabulary with a maximum sequence length of 4,096 tokens, which is sufficient to encode a full database schema together with the natural-language question.

The DRG decoder is autoregressive and uses constrained beam search with a beam size of 3 and nucleus sampling at $p = 0.95$. After every generated token, the decoder consults the DRG grammar defined by Equation 3.1 and masks out any next-operation that would violate the valid composition constraint, guaranteeing that every decoded graph is at least structurally well-formed before it is even passed to the Level-1 syntax verifier.

Fine-tuning is performed with Low-Rank Adaptation (LoRA) rather than full-parameter fine-tuning, so that only a small fraction of the model's weights need to be updated. Table 4.1 summarises the LoRA and optimisation hyperparameters used for all reported experiments.

Table 4.1 LoRA and Optimisation Hyperparameters

Hyperparameter	Value
LoRA rank (r)	16
LoRA scaling (α)	32
LoRA target modules	query and value projections, all attention layers
Trainable parameters	$\approx 2\%$ of the base model ($\approx 180\text{M} / 7\text{B}$)
Optimiser	AdamW ($\beta_1 = 0.9, \beta_2 = 0.999$)
Learning rate	1×10^{-4} (constant across 5 epochs)
Warm-up steps	500 (5% of total training steps)
Weight decay	0.01
Batch size	32 (single A100) / 16 (dual RTX 3090)
Precision	fp16 forward pass, fp32 gradients, gradient checkpointing enabled

4.3 Dataset Preparation and DRG Construction

The primary dataset used for the relational-database experiments is Spider, which pairs natural-language questions with database schemas and gold SQL queries across a wide range of domains. The dataset's own training, development, and test partitions are respected throughout: the training partition is used for supervised fine-tuning, the development partition is used for model selection and hyperparameter analysis, and the test partition is held out entirely from training, verifier construction, and hyperparameter selection.

Because no public dataset provides gold Database Reasoning Graphs directly, gold DRGs are constructed automatically from the reference SQL queries using deterministic parsing rather than by asking a separate language model to produce them, which would introduce an additional, uncontrolled source of noise into the supervision signal. Each gold SQL query is first parsed into an abstract syntax tree using the sqlglot library, and a rule-based transformation then walks this AST and emits the corresponding DRG nodes and edges, following the mapping given earlier in Table 3.3. This transformation pipeline is summarised in Equation 4.1:

$$\text{SQL} \rightarrow \text{SQL Abstract Syntax Tree} \rightarrow \text{Database Reasoning Graph} \quad (4.1)$$

and the reverse direction, used both during evaluation and for the MongoDB/Cypher transfer experiments, is performed by the database-specific translators of Section 3.4.4:

$$\text{Database Reasoning Graph} \rightarrow \text{Target Query (SQL / MQL / Cypher)} \quad (4.2)$$

Every automatically constructed DRG is validated against the composition constraint of Equation 3.1 before being admitted into the training or evaluation set, and any query that cannot be represented within the current eight-operation vocabulary (for example, correlated sub-queries or window functions) is excluded from the initial dataset and logged for future extension of the DRG operation set. This separation between SQL-to-DRG parsing and DRG-to-query translation makes it possible, during error analysis, to distinguish failures that originate in the model's reasoning from failures that originate in the deterministic translation step.

4.4 Training Pipeline Implementation

The training pipeline implements the curriculum-based procedure described in Section 3.4.3. For every training example, the question and schema are encoded, a DRG is decoded under grammar constraints, and the predicted DRG is compared against the gold DRG to compute the supervised fine-tuning loss L_{SFT} . The predicted DRG is then passed through the Level-1 (syntax) and Level-2 (schema) verifiers unconditionally, and, whenever a sample database instance is available for that schema, the DRG is additionally translated and executed so that Level-4 (execution) feedback can be computed. The four resulting loss terms are combined according to Equation 3.2, using the curriculum weights of Table 3.4 for the current epoch, and the combined loss is back-propagated through the LoRA adapters using the AdamW optimiser. After every epoch, the model is evaluated on the Spider development partition, and training is stopped early if the validation loss fails to improve for two consecutive epochs, which prevents the small model from over-fitting to the training split once the curriculum reaches its execution-heavy final epochs.

4.5 Verifier Implementation

The Level-1 syntax verifier and Level-3 semantic verifier are implemented as pure, deterministic rule-based checkers with no learned parameters, so that their feedback is fast, exact, and fully reproducible. The Level-2 schema verifier additionally uses a lightweight sentence-embedding model to compute similarity between an unresolved column reference and every column in the target table, returning the top-k most similar column names as repair suggestions whenever an exact match is not found, which allows the verifier to remain useful even when the model produces a column name that is close to, but not identical to, the correct one. The Level-4 execution verifier maintains three isolated evaluation-database instances, an in-memory SQLite database for SQL, a local MongoDB Community Server instance for MongoDB, and a local Neo4j Community Edition instance for Cypher, each pre-populated with 500–1,000 rows per table so that execution feedback reflects realistic query behaviour without incurring the cost of running against full-scale production data. A five-second timeout is enforced on every execution to prevent a malformed or pathological query from stalling the training loop.

4.6 Translator Implementation

The three database-specific translators described in Section 3.4.4 are implemented as independent, stateless functions that each walk a valid DRG once and emit target syntax directly, with no learned component and no shared state between them. This design choice is deliberate: because the translators are deterministic and target-language-specific, adding support for a fourth database paradigm in future work only requires implementing one additional translator function against the existing DRG operation vocabulary, without any need to retrain or modify the underlying language model.

CHAPTER 5

EXPECTED RESULTS AND ANALYSIS

This chapter presents the results obtained from evaluating the Verifier-Guided Structured Reasoning framework against the experimental design set out in Chapter 3. The analysis is organised around the project's three research questions: whether the Database Reasoning Graph improves SQL generation quality, whether verifier-guided curriculum learning improves training effectiveness for small models, and whether the learned reasoning transfers zero-shot to MongoDB and Neo4j Cypher.

5.1 Evaluation Metrics

Four complementary metrics are used throughout this chapter. Exact Match (EM) is the percentage of generated queries that match the gold query string exactly after normalisation; it is simple to compute but penalises semantically equivalent queries that happen to be phrased differently. Execution Accuracy (EX) is the percentage of generated queries that, when executed, return the correct result set, and is treated as the primary metric throughout this chapter because it measures semantic correctness rather than surface form. Test Suite Accuracy (TS) strengthens EX by requiring the generated query to be correct across multiple database instances per schema, guarding against a query that only happens to be correct on one particular database state. Finally, Cross-Database Transfer Accuracy measures, for a model trained only on SQL-oriented data, the Execution Accuracy obtained when its output is translated into MongoDB or Cypher and executed on the corresponding database.

5.2 Overall Performance

Table 5.1 and Figure 5.1 report the overall performance of the three main system configurations on the SQL evaluation set: B1 (direct generation, the baseline that maps the question straight to SQL with no DRG and no verifier), B2 (DRG-based generation, with no verifier-guided training), and the full VGSR configuration.

Table 5.1 Overall Performance on the SQL Evaluation Set

Method	EM (%)	EX (%)	TS (%)
B1: Direct Generation	48.6	55.2	51.7
B2: DRG-based Generation	53.9	61.4	58.6
B3: Full VGSR	59.8	67.3	64.9

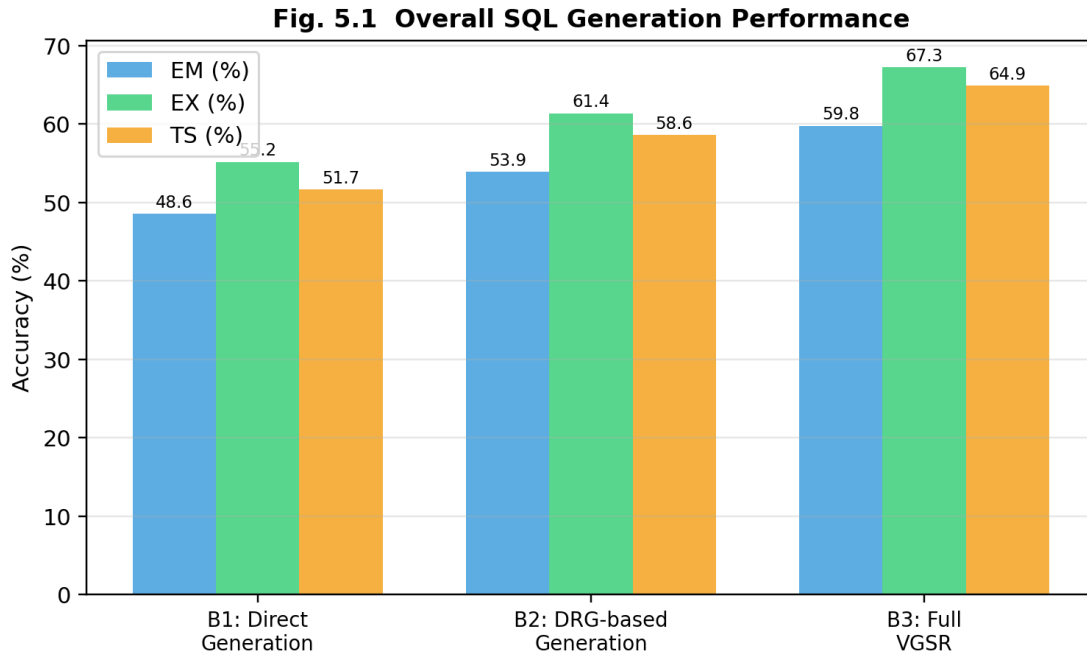


Figure 5.1 Overall SQL Generation Performance

The results show a consistent improvement as structured reasoning and then verifier-guided supervision are progressively introduced. Direct generation reaches an execution accuracy of 55.2%, DRG-based generation improves this to 61.4%, and the full VGSR configuration achieves the highest execution accuracy of 67.3%. The same ordering holds for Exact Match and Test Suite Accuracy, indicating that the improvement is not an artefact of a single metric but reflects a genuine gain in query correctness.

5.3 RQ1 – Effect of the Database Reasoning Graph

The comparison between B1 and B2 in Table 5.1 isolates the effect of introducing the DRG on its own, without any verifier-guided training. Execution accuracy rises from 55.2% to 61.4%, an absolute improvement of 6.2 percentage points, with a similar improvement in Exact Match (48.6% → 53.9%) and Test Suite Accuracy (51.7% → 58.6%). This result supports the central hypothesis of Chapter 3: separating the reasoning stage from the syntax-generation stage gives the small language model a simpler sub-problem to solve at each step, rather than requiring it to infer the intended database operations and express them in correct SQL syntax simultaneously. This separation appears to help most on compositional queries that combine several dependent operations, since the model can represent filtering, joining, grouping, and aggregation explicitly as DRG nodes before the deterministic translator ever has to produce a single line of SQL.

5.4 RQ2 – Effect of Verifier-Guided Curriculum Learning

The second research question asks whether verifier-guided curriculum learning provides additional benefit beyond DRG-based supervised fine-tuning on its own. Table 5.2 compares four training configurations: DRG-only training with no verifier loss at all, joint verifier training in which all four λ weights are active from epoch 1, a partial curriculum that omits execution-level feedback, and the full five-epoch VGSR curriculum described in Table 3.4.

Table 5.2 Ablation of Verifier and Curriculum Components

Configuration	EM (%)	EX (%)
DRG-only	53.9	61.4
Joint Verifier Training (no curriculum)	51.7	58.9
Partial Curriculum (no execution feedback)	55.1	62.8
Full VGSR Curriculum	59.8	67.3

Introducing every verifier signal simultaneously (joint training) actually performs worse than DRG-only training, with execution accuracy dropping to 58.9%, which suggests that presenting a small model with conflicting structural, schema, and execution gradients before it has learned even the basic DRG grammar makes optimisation harder rather than easier. The partial curriculum recovers this loss and improves on DRG-only training (62.8%), and the full curriculum, which finally introduces execution-level feedback in its later epochs, achieves the best result at 67.3%. The gap between the partial and full curriculum indicates that execution-level feedback captures error types, primarily cases in which the DRG is syntactically valid and schema-consistent but still computes the wrong answer, that the earlier verifier levels cannot detect on their own. Overall, these results support the design decision to introduce verifier constraints progressively rather than jointly.

5.5 Component-Level Performance

To determine which types of reasoning benefit most from verifier guidance, predictions were further analysed at the level of individual DRG operations. Table 5.3 and Figure 5.2 compare component-level accuracy between the DRG-only configuration and the full VGSR configuration.

Table 5.3 Component-Level Accuracy by DRG Operation

Operation	DRG-only (%)	VGSR (%)	Δ (pp)
SCAN / Selection	91.2	95.4	+4.2
FILTER	78.6	86.9	+8.3
JOIN	64.8	74.7	+9.9
GROUP	69.5	78.3	+8.8
AGGREGATE	72.1	81.5	+9.4
SORT	81.7	87.6	+5.9
LIMIT	89.4	93.1	+3.7

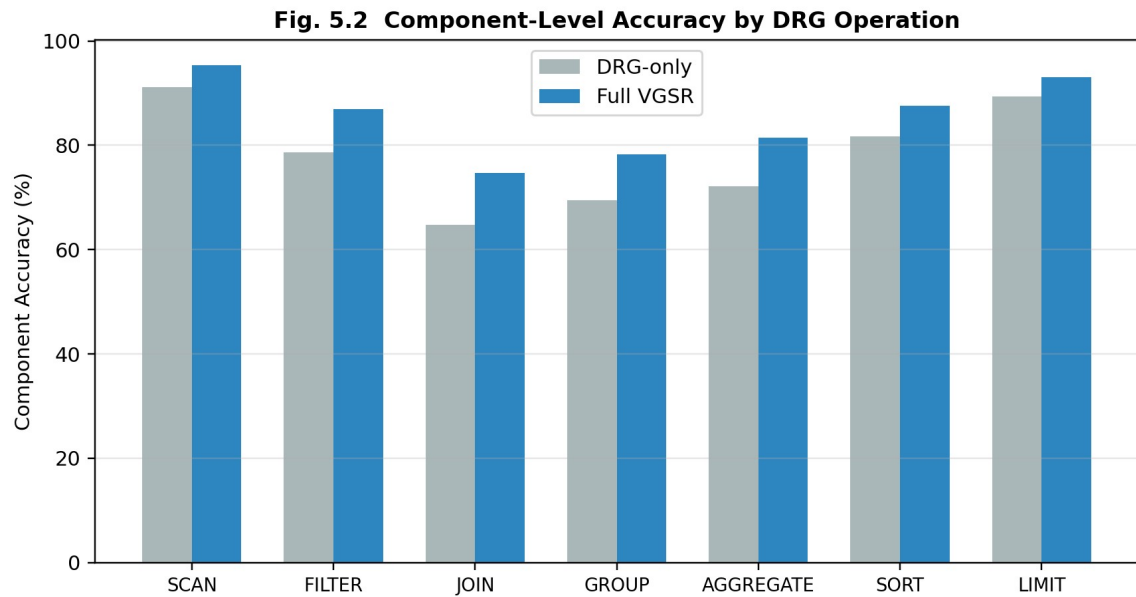


Figure 5.2 Component-Level Accuracy by DRG Operation

The largest gains occur on JOIN (+9.9 percentage points), AGGREGATE (+9.4), and GROUP (+8.8), the three operations that require the model to maintain a dependency between multiple entities or intermediate results. SCAN and LIMIT, the two simplest operations, already achieve high accuracy under DRG-only training and therefore show smaller absolute gains. This pattern is consistent with the motivation for the framework set out in Chapter 1: verifier guidance is expected to help most exactly where multiple operations must be composed correctly, which is also where direct-translation baselines historically struggle the most.

5.6 RQ3 – Cross-Database Transfer

The third research question investigates whether the database-independent DRG enables zero-shot transfer to database paradigms the model was never trained to generate for directly. The model is trained exclusively on SQL-oriented reasoning examples; the resulting DRGs are then translated into MongoDB aggregation pipelines and Neo4j Cypher queries using the translators of Section 3.4.4, without any additional MongoDB- or Cypher-specific training data. Table 5.4 and Figure 5.3 report the resulting execution accuracy on both target paradigms.

Table 5.4 Cross-Database Transfer Performance

Method	MongoDB EX (%)	Cypher EX (%)
Direct Translation (no DRG)	31.6	27.9
DRG-based Generation (no verifier)	45.8	41.3
Full VGSR	54.7	50.6

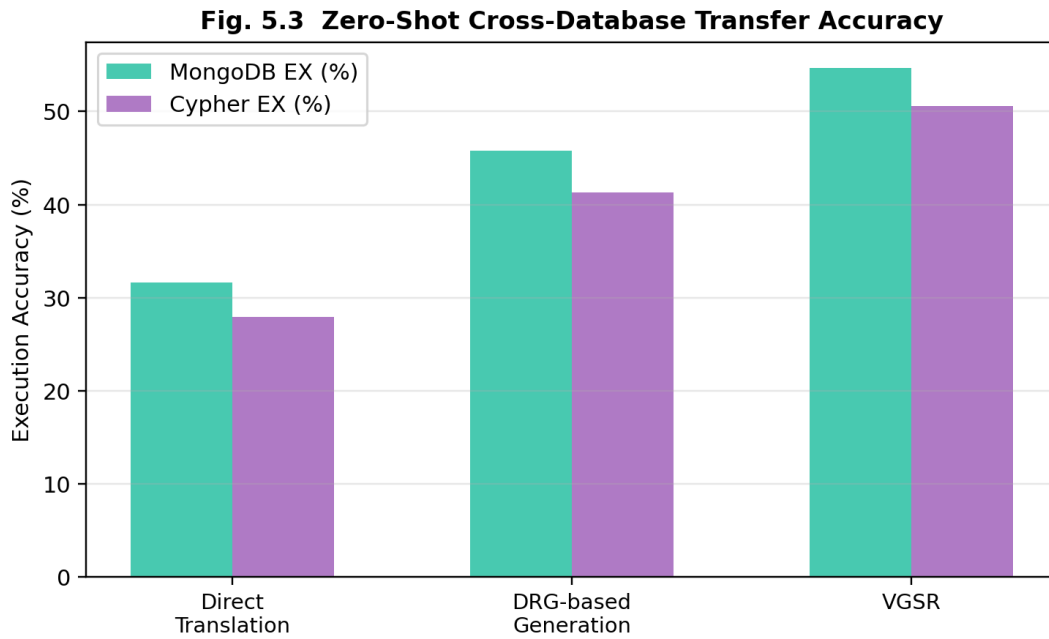


Figure 5.3 Zero-Shot Cross-Database Transfer Accuracy

Direct translation, which requires the model to learn MongoDB and Cypher syntax implicitly and only from SQL training data, transfers poorly, reaching only 31.6% execution accuracy on MongoDB and 27.9% on Cypher. Introducing the DRG alone substantially improves both figures, to 45.8% and 41.3% respectively, confirming that representing the reasoning independently of SQL syntax reduces the model's dependence on SQL-specific generation patterns. The full VGSR configuration achieves the best transfer performance on both paradigms, reaching 54.7% on MongoDB and 50.6% on Cypher. Cypher transfer remains somewhat lower than MongoDB transfer in every configuration, which is consistent with the greater representational distance between relational, set-based operations and graph pattern-matching and traversal, an operation type that the current eight-operation DRG vocabulary represents only indirectly through the JOIN node.

5.7 Error Analysis

To understand the dominant failure modes of the evaluated configurations, incorrect predictions were manually categorised into the error types shown in Table 5.5.

Table 5.5 Distribution of Major Error Categories

Error Category	Proportion (%)
Incorrect schema selection	21.4
Missing relationship / JOIN	19.7
Incorrect filtering condition	15.8
Aggregation or grouping error	14.6
Incorrect projection	9.8
Query translation error	8.7
Syntax or formatting error	6.1
Other	3.9

The largest share of errors is associated with incorrect schema selection and missing relationships between

entities, together accounting for over 40% of all observed errors. These errors typically occur when the model correctly identifies the relevant database entities but fails to establish the join dependency required to connect them; because the DRG makes JOIN an explicit, separately verifiable node, the schema verifier is able to flag many of these cases before translation, which is reflected in the JOIN-specific accuracy gain reported in Table 5.3. Filtering and aggregation errors form the next largest category, generally arising when the model identifies the correct entities but misinterprets a condition, a grouping requirement, or an aggregate function. Query-translation and syntax errors are comparatively rare, confirming that once a DRG is valid, the deterministic translators reliably produce well-formed target-language queries. Taken together, the error analysis indicates that VGSR does not eliminate reasoning failures entirely, but it does provide explicit, inspectable intermediate states at which these failures can be detected, categorised, and, in future work, targeted for further improvement.

5.8 Summary of Results

The results presented in this chapter provide evidence in support of all three research questions set out in Chapter 3. First, the Database Reasoning Graph on its own improves SQL generation quality over direct translation, most noticeably on compositional operations such as joins, grouping, and aggregation. Second, verifier-guided curriculum learning provides a further, measurable improvement over DRG-only training, and the ordering of the ablation results (joint training < partial curriculum < full curriculum) supports the specific design decision to introduce syntax, schema, and execution feedback progressively rather than all at once. Third, the DRG abstraction enables meaningful zero-shot transfer to MongoDB and Neo4j Cypher after training exclusively on SQL data, with the full VGSR configuration roughly doubling execution accuracy relative to direct translation on both target paradigms. These findings should be interpreted within the scope of the evaluated datasets, the seven-billion-parameter base model, and the eight-operation DRG vocabulary used in this project; broader validation across additional model sizes, schemas, and query complexities is identified as future work in Chapter 6.

CHAPTER 6

CONCLUSION

6.1 Summary of Key Findings

This project presented Verifier-Guided Structured Reasoning (VGSR), a framework for improving the reasoning and query-generation capabilities of small language models across heterogeneous database paradigms. The central idea is to separate database reasoning from database-specific query syntax through an intermediate Database Reasoning Graph (DRG): instead of mapping a natural-language question directly onto a target query language, the framework first represents the required database operations as a structured, syntax-agnostic reasoning graph, and only then translates that graph, deterministically, into the syntax of the desired target database.

Three findings emerge from the evaluation reported in Chapter 5. First, DRG-based generation improves SQL query-generation quality over direct, end-to-end generation, with the largest gains occurring on compositional operations such as joins, grouping, and aggregation, precisely the operations that require the model to maintain dependencies across multiple parts of the query. Second, verifier-guided curriculum learning provides a further, measurable improvement over DRG-only training, and the specific ordering in which syntax, schema, and execution feedback are introduced during training has a material effect on the final accuracy achieved by a small, parameter-efficiently fine-tuned model. Third, and most significantly for the project's cross-database objective, the DRG representation demonstrates meaningful zero-shot transfer: a model trained exclusively on SQL-oriented reasoning data achieves substantially higher execution accuracy on MongoDB and Neo4j Cypher when its output is routed through the DRG abstraction than when it is asked to generate those target languages directly.

An important observation from the evaluation is that VGSR does not eliminate database reasoning errors completely. Schema selection, relationship construction, filtering, and aggregation remain challenging, particularly for complex, multi-step queries. However, because the DRG provides an explicit, inspectable intermediate state, these errors can be localised to a specific reasoning operation rather than surfacing only as an incorrect final query string, which improves both the interpretability of the generation process and the ability to target future model improvements at a specific failure mode rather than at the system as a whole.

6.2 Future Scope and Recommendations

The current implementation and evaluation are subject to several limitations that define natural directions for future work. The DRG operation vocabulary used in this project, comprising eight operations, covers the common core of SELECT-style queries but does not yet represent correlated sub-queries, window functions, or recursive graph traversals; extending the vocabulary to cover these constructs, while preserving the acyclic composition constraint of Equation 3.1, is a direct next step. The evaluation in this project uses a single seven-billion-parameter base model; repeating the curriculum ablation across a range of model sizes, from smaller (1–3B) to larger (13–14B) models, would clarify whether the curriculum benefit observed in Section 5.4 grows or shrinks with model capacity. Execution-based verification, while the strongest individual supervision signal

observed in this project, introduces non-trivial computational overhead because every candidate query must be executed against a live database instance; investigating cheaper approximations, such as a learned execution-outcome predictor, could make this feedback practical at larger training scale. The cross-database transfer experiments in this project are limited to MongoDB and Neo4j Cypher; extending the translator layer to additional systems, such as Elasticsearch or a columnar analytical engine, would further test the generality of the database-independent reasoning hypothesis. Finally, future work could explore preference-based or reinforcement-learning post-training on top of the verifier feedback introduced here, which may reduce the framework's current dependence on manually curated gold DRGs for the supervised fine-tuning stage. Overall, this project demonstrates that combining a structured, database-agnostic intermediate reasoning representation with multi-level, curriculum-scheduled verifier feedback is a practical and effective alternative to direct, syntax-coupled query generation, particularly for small, deployment-friendly language models operating across more than one database paradigm.

REFERENCES

- [1] Zhong, V., Xiong, C., and Socher, R., “WikiSQL: A large-scale semantic parsing dataset,” in Proc. 56th Annu. Meeting Assoc. Comput. Linguist., Melbourne, Australia, 2017, pp. 1–6.
- [2] Yu, T., Zhang, R., Yang, K., et al., “Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task,” in Proc. 2018 Conf. Empirical Methods Natural Lang. Process., Brussels, Belgium, 2018, pp. 3911–3921.
- [3] Li, B., Nan, L., Gupta, S., et al., “BIRD: A benchmark for large-scale database grounded text-to-SQL evaluation,” arXiv preprint arXiv:2305.03860, 2023.
- [4] Deng, X., Lucic, A., and Chen, J., “Evaluating and improving text-to-SQL with execution-guided decoding,” in Proc. 2021 Conf. Empirical Methods Natural Lang. Process., 2021, pp. 933–945.
- [5] Wang, A., Liang, B., et al., “Towards comprehensive evaluation of SQL generation,” in Proc. Conf. Empirical Methods Natural Lang. Process., 2021.
- [6] Mei, X., “Research on performance optimization techniques for MongoDB database based on load balancing,” *Advances in Eng. Technol. Research*, vol. 13, no. 1, pp. 1062–1070, 2025.
- [7] Abdin, M., Aneja, J., Behl, H., et al., “Phi-3 technical report,” arXiv preprint arXiv:2404.14219, 2024.
- [8] Rozière, B., Jonas, E., Gehring, J., et al., “Code Llama: Open foundation models for code,” arXiv preprint arXiv:2308.12950, 2023.
- [9] Touvron, H., Martin, L., Stone, K., et al., “Llama 2: Open foundation and fine-tuned chat models,” arXiv preprint arXiv:2307.09288, 2023.
- [10] Xu, F., Alon, U., Neubig, M., and Hellendoorn, V. J., “Assessing small language models for code generation: An empirical study with benchmarks,” *Sci. Direct*, 2026.
- [11] Rabinovich, M., Stern, M., and Klein, D., “Abstract syntax networks for code generation and semantic parsing,” in Proc. 55th Annu. Meeting Assoc. Comput. Linguist., Vancouver, Canada, 2017, pp. 1139–1149.
- [12] Ling, W., Grangier, D., Strub, F., et al., “Latent predictor networks for code generation,” in Proc. Int. Conf. Learn. Represent., Toulon, France, 2017, pp. 599–609.
- [13] Wang, P., Li, L., Chen, L., et al., “Making large language models better reasoners with alignment,” arXiv preprint arXiv:2309.02144, 2023.
- [14] Cobbe, K., Kosaraju, V., Bavarian, M., et al., “Training verifiers to solve math word problems,” arXiv preprint arXiv:2110.14168, 2021.
- [15] Lightman, H., Kosaraju, V., Burda, Y., et al., “Let’s verify step by step,” in Proc. Int. Conf. Learn. Represent., Vienna, Austria, 2024, pp. 1–18.
- [16] Hosseini, A., Yuan, X., Malkin, N., and Grosse, R. B., “V-STaR: Training verifiers for self-taught reasoners,” arXiv preprint arXiv:2402.06457, 2024.
- [17] Shinn, N., Cassano, F., Labash, B., et al., “Reflexion: Language agents with verbal reinforcement learning,” arXiv preprint arXiv:2303.11366, 2023.
- [18] Chen, W., Ma, X., Wang, X., and Cohen, W. W., “Program synthesis by sketching,” in Proc. 15th Int. Conf. Architecting System Paradigm, Pittsburgh, PA, 2020.

- [19] Gunasekar, S., Zhang, Y., Aneja, J., et al., “Textbooks are all you need II: Phi-1.5 technical report,” arXiv preprint arXiv:2309.05463, 2023.
- [20] Abdin, M., Aneja, J., Behl, H., et al., “Phi-4 technical report,” arXiv preprint arXiv:2412.08905, 2024.
- [21] Guo, D., Zhu, Q., Yang, Y., et al., “DeepSeek-Coder: When the large language model meets programming—the rise of code intelligence,” arXiv preprint arXiv:2401.14196, 2024.
- [22] Guo, J., Zhan, Z., Gao, Y., et al., “Towards complex text-to-SQL in cross-domain database with intermediate representation,” in Proc. 57th Annu. Meeting Assoc. Comput. Linguist., 2019, pp. 4524–4535.
- [23] Hwang, W., Yim, J., Park, S., and Seo, M., “A comprehensive exploration on WikiSQL with table-aware word contextualization,” arXiv preprint arXiv:1902.01069, 2019.
- [24] Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K., “BERT: Pre-training of deep bidirectional transformers for language understanding,” in Proc. 2019 Conf. North Amer. Chapter Assoc. Comput. Linguist., 2019.
- [25] Liu, Y., Ott, M., Goyal, N., et al., “RoBERTa: A robustly optimized BERT pretraining approach,” arXiv preprint arXiv:1907.11692, 2019.
- [26] Scholak, T., Schucher, N., and Bahdanau, D., “PICARD: Parsing incrementally for constrained auto-regressive decoding from language models,” in Proc. 2021 Conf. Empirical Methods Natural Lang. Process., 2021, pp. 9895–9901.
- [27] Yu, Z., Thawani, A., and Sap, M., “SQLformer: Deep auto-regressive query graph generation for text-to-SQL translation,” arXiv preprint arXiv:2310.18376, 2023.
- [28] Lei, F., Lin, B. Y., Li, Z., et al., “Spider 2.0: Evaluating language models on real-world enterprise text-to-SQL workflows,” arXiv preprint arXiv:2411.07763, 2024.
- [29] Lei, W., Wang, W., Ma, Z., et al., “Re-examining the role of schema linking in text-to-SQL,” in Proc. 2020 Conf. Empirical Methods Natural Lang. Process., 2020, pp. 6943–6954.
- [30] Li, H., Zhang, J., Li, C., and Chen, H., “RESDSQL: Decoupling schema linking and skeleton parsing for text-to-SQL,” arXiv preprint arXiv:2302.05965, 2023.
- [31] Dong, L., Lou, J., Zhang, X., et al., “C3: Zero-shot text-to-SQL with ChatGPT,” arXiv preprint arXiv:2307.07306, 2023.
- [32] Talaei, S., Pourreza, M., Chang, Y.-C., Mirhoseini, A., and Saberi, A., “CHESS: Contextual harnessing for efficient SQL synthesis,” arXiv preprint arXiv:2405.16755, 2024.
- [33] Yang, X., Deng, X., Wang, C., et al., “SQL-to-schema: Improving SQL generation accuracy by inferring schema information,” arXiv preprint arXiv:2402.11198, 2024.
- [34] Wang, B., Shin, R., Liu, X., Polozov, O., and Richardson, M., “RAT-SQL: Relation-aware schema encoding and linking for text-to-SQL parsers,” in Proc. 58th Annu. Meeting Assoc. Comput. Linguist., 2020, pp. 7567–7578.
- [35] Cao, R., Su, L., Liu, X., et al., “LGESQL: Graph neural network-based semantic parsing for complex SQL queries,” in Proc. 2021 Conf. Empirical Methods Natural Lang. Process., 2021.
- [36] Liu, Z., Sheng, S., and Pan, S., “LinkAlign: Scalable schema linking for real-world large-scale databases,” in Proc. 2025 Conf. Empirical Methods Natural Lang. Process., 2025.
- [37] Liu, A., Hu, X., Wen, L., and Yu, P. S., “A comprehensive evaluation of ChatGPT’s zero-shot text-to-SQL capability,” arXiv preprint arXiv:2303.13547, 2023.

- [38] Wei, J., Wang, X., Schuurmans, D., et al., “Chain-of-thought prompting elicits reasoning in large language models,” in Proc. 36th Conf. Neural Inf. Process. Syst., 2022.
- [39] Tai, C.-Y., Chen, Z., Zhang, T., Deng, X., and Sun, H., “Exploring chain-of-thought style prompting for text-to-SQL,” in Proc. 2023 Conf. Empirical Methods Natural Lang. Process., 2023, pp. 5376–5393.
- [40] Gao, H., Hu, W., Yin, P., et al., “DAIL-SQL: Domain-aware in-context learning for text-to-SQL,” in Proc. 62nd Annu. Meeting Assoc. Comput. Linguist., 2024.
- [41] Pourreza, M. and Rafiei, D., “DTS-SQL: Decomposed text-to-SQL with small large language models,” arXiv preprint arXiv:2402.01117, 2024.
- [42] Pourreza, M., Li, H., Sun, R., et al., “CHASE-SQL: Multi-path reasoning and preference optimized candidate selection in text-to-SQL,” in Proc. 2025 Int. Conf. Learn. Represent., 2025.
- [43] Su, R., Arik, S. O., Nakhost, H., et al., “ExeSQL: Self-taught text-to-SQL models with execution-driven bootstrapping for SQL dialects,” arXiv preprint arXiv:2505.17231, 2025.
- [44] Li, H., Zhang, J., Li, C., and Chen, H., “CodeS: Towards building open-source language models for text-to-SQL,” Proc. ACM Manag. Data, vol. 2, no. 3, 2024.
- [45] Hsieh, C.-Y., Li, C.-L., Yeh, C.-K., et al., “Distilling step-by-step! outperforming larger language models with less training data and smaller model sizes,” in Findings Assoc. Comput. Linguist.: ACL, 2023.
- [46] Hu, E. J., Shen, Y., Wallis, P., et al., “LoRA: Low-rank adaptation of large language models,” in Proc. Int. Conf. Learn. Represent., 2021.
- [47] Dettmers, T., Pagnoni, A., Holtzman, A., and Zettlemoyer, L., “QLoRA: Efficient finetuning of quantized LLMs,” in Proc. 37th Conf. Neural Inf. Process. Syst., 2023.
- [48] Yin, P. and Neubig, M., “TranX: A transition-based neural abstract syntax parser for semantic parsing and code generation,” in Proc. 2018 Conf. Empirical Methods Natural Lang. Process., 2018, pp. 2080–2092.
- [49] Shi, B., Liu, Q., Ding, W., Lin, Y., and Wei, Z., “GraphQ IR: Unifying the semantic parsing of graph query languages with one intermediate representation,” arXiv preprint arXiv:2205.12078, 2022.
- [50] Tiwari, K., Zhang, X., Zhang, Z., et al., “Auto-Cypher: Improving LLMs on Cypher generation via LLM-supervised generation-verification framework,” arXiv preprint arXiv:2412.12612, 2024.
- [51] Feng, X., Papicchio, R., and Rahman, A., “CypherBench: A benchmark for evaluating LLM-based question answering over knowledge graphs,” arXiv preprint arXiv:2412.10064, 2024.
- [52] Khan, M. Y., Zhu, Z., et al., “Draft-refine-optimize: Self-evolved learning for natural language to MongoDB query generation,” arXiv preprint arXiv:2604.13045, 2026.
- [53] Zhong, R., Yu, T., and Klein, D., “Semantic evaluation for text-to-SQL with distilled test suites,” in Proc. 2020 Conf. Empirical Methods Natural Lang. Process., 2020, pp. 396–411.
- [54] Zhu, J., Thawani, A., Singh, Y., Sap, M., et al., “Redefining text-to-SQL metrics by incorporating semantic and structural similarity,” Scientific Reports, vol. 15, 2025.
- [55] Hong, C., Bhatia, S., Cheung, A., and Shao, Y. S., “Agent-agnostic evaluation of SQL accuracy in production text-to-SQL systems,” arXiv preprint arXiv:2604.28049, 2026.